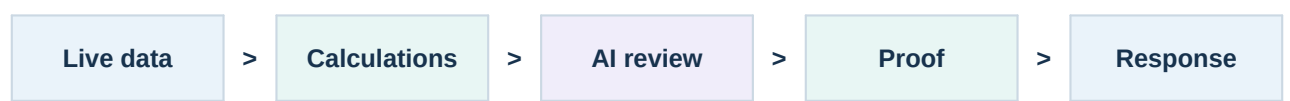


Stock Analyser Backend

A complete, easy-to-understand project handbook

THE PROJECT IN ONE SENTENCE
A hybrid stock-research backend that combines live Yahoo Finance evidence, reproducible technical rules, specialized Gemini agents, risk review, backtesting, authentication, and proof for every result.



Built with FastAPI, Python, Yahoo Finance, CrewAI, Gemini, SQLAlchemy, SQLite, JWT and deterministic backtesting.

Purpose: project revision, technical review, frontend planning and interview preparation

Contents

Section	Topic
1	What the system does
2	Complete input-to-response flow
3	Market data and technical indicators
4	Signal and risk calculation
5	What the AI agents do
6	How results are verified
7	Backtesting engine
8	API, background jobs and frontend flow
9	Authentication and database
10	Technology choices
11	Limitations and future improvements
12	Interview questions and answers
13	Quick glossary and revision sheet

HOW TO READ THIS HANDBOOK

Start with Sections 1 and 2 for the big picture. Use Sections 3-10 to understand implementation decisions. Revise Sections 12 and 13 before an interview.

1. What the system does

The user provides a stock ticker such as AAPL. The backend collects current market evidence, calculates technical indicators, creates a reproducible technical signal, asks specialized AI agents to research and challenge that signal, and returns a response with sources and calculation proof.

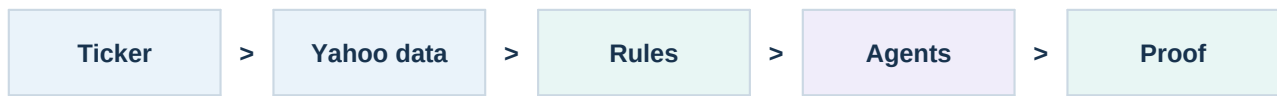
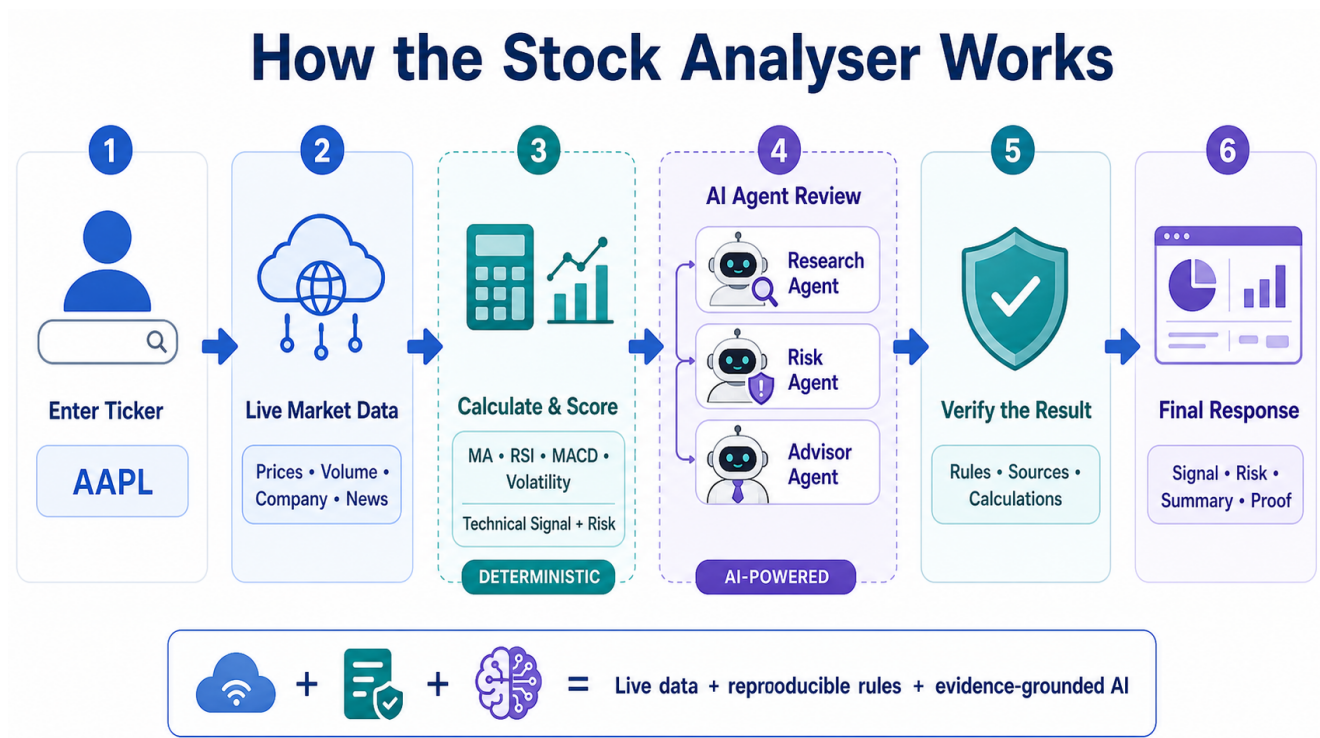
- It is a research and decision-support tool, not an automatic trading system.
- BUY, HOLD and SELL are technical signals, not guaranteed outcomes or personalized advice.
- Stock-specific prices and results are calculated from data; they are not fixed in the code.
- Strategy thresholds are explicit configuration so the same logic can be reproduced and backtested.

Hybrid design

Part	Responsibility	Why
Yahoo Finance	Supplies prices, volume, company details and attributable news	Provides external evidence
Python rules	Calculates indicators, signal and risk	Reproducible and testable
AI agents	Prioritize research, challenge risk and select relevant evidence	Adds context without inventing numbers
Proof layer	Shows formulas, thresholds, sources and score contributions	Makes the result auditable

IMPORTANT
The system is intelligent but controlled. The AI agents cannot silently change the calculated signal, risk level or market numbers.

2. Complete input-to-response flow



Step	Stage	What happens
1	Input	The user submits a validated ticker such as AAPL.
2	Collection	Python uses yfinance to obtain adjusted prices, volume, company metadata and recent news.
3	Calculation	The backend calculates MA20, MA50, RSI14, MACD, returns and annualized volatility.
4	Decision	Transparent scoring rules lock the technical action and risk classification.
5	Agent review	Research and Risk agents review in parallel; the Advisor selects the most relevant supported points.
6	Proof	Sources, timestamps, formulas, rule contributions and evidence quality are attached.
7	Delivery	The API returns JSON directly or completes a background job for the frontend.

3. Market data and technical indicators

Who collects the data?

The Python backend collects data. The agents do not browse for prices or invent numbers. This separation prevents LLM hallucination from entering numerical calculations.

Data	Meaning	Used for
Adjusted close	Closing price adjusted for splits and distributions	Indicators and returns
Volume	Number of shares traded	Liquidity context
Company metadata	Name, sector, industry, exchange	Research context
News	Title, publisher, URL and UTC publication time	Attributable research evidence

Indicators in plain English

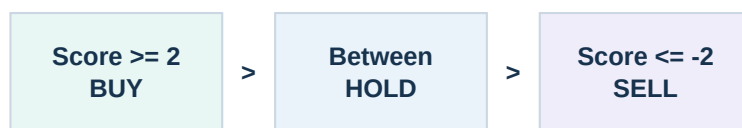
Term	Simple meaning	Interpretation
MA20	Average of the latest 20 adjusted closes	Short-term trend
MA50	Average of the latest 50 adjusted closes	Medium-term trend
RSI14	Compares recent upward and downward momentum on a 0-100 scale	Above 70 can be overbought; below 30 can be oversold
MACD	12-day exponential average minus the 26-day exponential average	Compared with its 9-day signal line to identify momentum
Volatility	How widely daily returns move	Higher values mean greater historical uncertainty
3-month return	Percentage price change over about 63 trading days	Recent performance context

WHY 252?
Markets have roughly 252 trading sessions in a typical year. Annualized volatility is daily return standard deviation x $\sqrt{252}$ x 100.

4. Signal and risk calculation

Technical signal scoring

Rule	Condition	Contribution
Moving-average trend	Price > MA20 > MA50	+2
Moving-average trend	Price < MA20 < MA50	-2
RSI	RSI >= configured overbought threshold (default 70)	-1
RSI	RSI <= configured oversold threshold (default 30)	+1
MACD	MACD > signal line	+1
MACD	MACD <= signal line	-1



AAPL example

Observation	Points
Bullish moving-average trend	+2
RSI 80.28: overbought warning	-1
MACD above its signal line	+1
Total	2 -> technical BUY

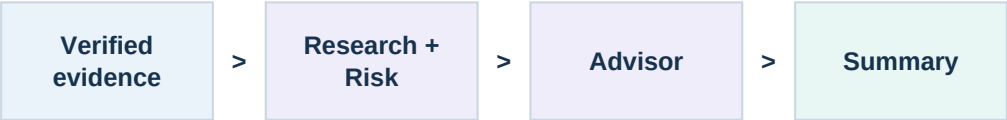
Risk classification

Annualized historical volatility	Risk
Below 20%	LOW
20% to below 40%	MEDIUM
40% or higher	HIGH

CONFIGURATION, NOT HIDDEN MAGIC

Default thresholds are centralized in `strategy_config.py` and can be overridden with environment variables. Live analysis and backtesting import the same configuration.

5. What the AI agents do



Agent	Question it answers	Allowed work
Research Agent	Which supplied company facts and news items matter most?	Select existing company fields, news indexes and caveats
Risk Agent	What could make the technical signal unreliable?	Select existing risk evidence and known data gaps
Advisor Agent	Which supported strengths and warnings should the user see first?	Select evidence IDs after considering both specialist reviews

What agents cannot do

- Invent a price, ratio, indicator or news article.
- Change the locked BUY/HOLD/SELL action.
- Change the locked LOW/MEDIUM/HIGH risk.
- Add unsupported factual prose to the response.

WHY USE AGENTS IF DATA IS ALREADY COLLECTED?
Data collection answers 'what are the facts?' Agents answer 'which facts matter?', 'what could go wrong?' and 'what should be presented first?'.

How the summary is produced

Agents select the relevant strength and warning. The backend then assembles controlled wording from those verified statements. This is AI-assisted and evidence-grounded, rather than unrestricted text generation.

6. How results are verified

Layer	Verification
1. Input	Ticker normalization and character validation
2. Data	Non-empty market history and minimum required trading days
3. Numbers	Finite-value checks for calculated metrics
4. Arithmetic	Every rule contribution is returned and must add to the total score
5. Boundaries	The score and volatility are checked against published thresholds
6. Agent output	Only allowed evidence IDs, company fields, news indexes and gap codes are accepted
7. Provenance	Source URL, UTC timestamp, period, trading-day count, cache status and rule version
8. Tests	Automated tests cover calculations, API errors, auth, storage, jobs and backtesting behavior

Understanding evidence quality

The response uses `evidence_quality.score`. It measures whether expected metrics, company metadata, attributable news and agent reviews were available. It is not a probability of profit and must never be displayed as investment certainty.

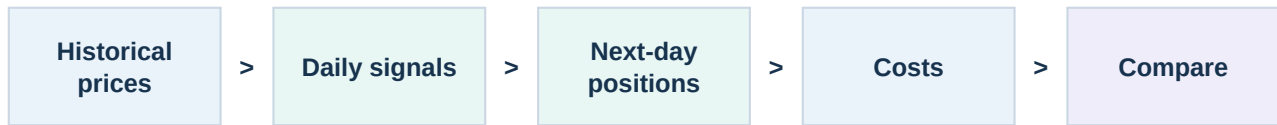
What verification does not mean

NO GUARANTEE

Verification proves that the input, calculation path and supporting evidence are traceable. It does not prove that a future trade will be profitable. Markets are uncertain and can change after the analysis timestamp.

7. Backtesting engine

Backtesting asks: 'How would these exact rules have behaved on historical data?' It does not ask an LLM to remember the past.



- The same strategy thresholds used by live analysis are applied to each historical day.
- A signal calculated at today's close becomes a position on the next trading day. This reduces look-ahead bias.
- Transaction costs are deducted when the simulated position changes.
- Performance is compared with buying and holding the same stock.
- The simple overview is returned first; technical detail and chart data are optional.

Metric	Plain meaning
Total return	Percentage gain or loss over the full test
Annualized return	Return expressed per year
Sharpe ratio	Return relative to volatility, using a 0% risk-free assumption
Maximum drawdown	Largest fall from a previous portfolio peak
Exposure	Percentage of time with a market position
Signal hit rate	How often actionable signals had the expected forward direction

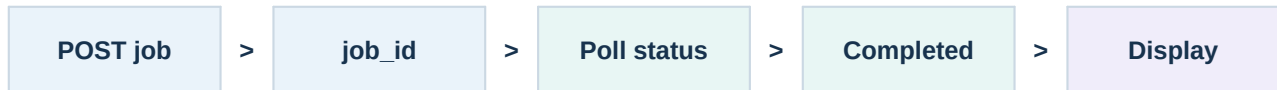
INTERPRETATION

A poor backtest is useful information. It shows that the rules should not be advertised as profitable and motivates walk-forward validation and strategy improvement.

8. API, background jobs and frontend flow

Why background jobs?

Agent analysis can take much longer than an ordinary API request. A background job returns an ID immediately, allowing the future frontend to show progress instead of appearing frozen.

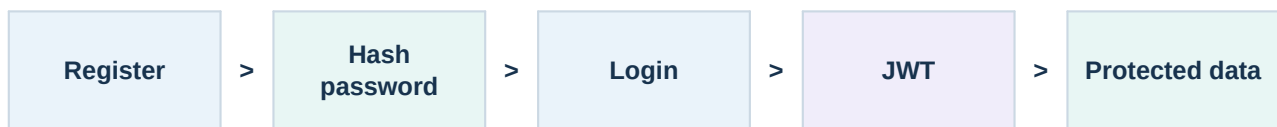


Endpoint	Purpose
GET /health	Deployment health check
POST /analyze	Synchronous analysis for direct testing
POST /analysis-jobs	Start asynchronous agent analysis
GET /analysis-jobs/{job_id}	Read queued, running, completed or failed state
POST /backtest	Run deterministic historical simulation
POST /auth/register	Create a local account
POST /auth/token	Receive a signed JWT access token
GET /auth/me	Verify the current authenticated user
/watchlist	Manage personal tickers
/saved-analyses and /saved-backtests	Store and review past results

Recommended frontend screens

- Login and registration
- Ticker search
- Analysis progress
- Simple signal and warning
- Expandable evidence and proof
- Backtesting dashboard
- Watchlist
- Saved history

9. Authentication and database



- Passwords are hashed with Argon2 and are never stored or returned as plain text.
- A successful login creates a signed JWT with a configurable expiration time.
- Protected endpoints identify the user from the verified token.
- SQLite stores users, watchlist items, saved analyses and saved backtests at zero infrastructure cost.
- SQLAlchemy separates application logic from the database engine, making a later MySQL switch configuration-based.

Current choice	Why it fits this project	Upgrade path
SQLite	Free, local, simple and enough for a single-user resume demo	Set a MySQL DATABASE_URL
In-memory jobs	No Redis service or cost for a one-instance demo	Use Redis/Celery only for multi-instance reliability
JWT	Demonstrates real authentication and prepares the frontend	Add refresh tokens/revocation only if required

DEPLOYMENT SECRETS

Set GEMINI_API_KEY and a long random JWT_SECRET in the hosting platform. Never commit the .env file or expose tokens in screenshots.

10. Technology choices

Technology	Role	Why chosen
Python	Backend and quantitative calculations	Strong data ecosystem and readable code
FastAPI	HTTP API and Swagger documentation	Validation, async-friendly design and automatic OpenAPI docs
yfinance	Yahoo Finance data access	Simple access to market history, company metadata and news
pandas / NumPy	Indicators and vectorized backtesting	Efficient time-series calculations
CrewAI	Multi-agent orchestration	Clear Research, Risk and Advisor roles
Google Gemini	Agent reasoning	Free-tier model option and CrewAI integration
Pydantic	Request validation	Typed, documented API inputs
SQLAlchemy	Database abstraction	SQLite today; MySQL-ready later
SQLite	Demo persistence	Zero-cost and no separate server
PyJWT	Signed access tokens	Standard bearer-token authentication
pwdlib / Argon2	Password hashing	Modern password protection
Uvicorn	ASGI server	FastAPI-compatible local and hosted runtime
unittest	Automated verification	Built-in, reliable and provider-free tests

WHY NOT LET THE LLM DO EVERYTHING?

Numerical finance outputs must be reproducible and backtestable. Deterministic code handles numbers; agents handle evidence prioritization, criticism and context.

11. Limitations and future improvements

Current limitations

- The signal mainly uses technical price indicators and is not a complete valuation model.
- Yahoo Finance is a convenient third-party source but can be delayed, rate-limited or incomplete.
- News selection does not yet perform full article-level sentiment or event extraction.
- In-memory jobs disappear when the process restarts and do not coordinate across multiple servers.
- SQLite storage can be ephemeral on some cloud hosting plans.
- Backtest results for one ticker or period do not establish general predictive quality.

Recommended roadmap

Priority	Improvement	Value
1	Walk-forward and multi-ticker validation	Tests stability across time and different stocks
2	Benchmark strategy versions	Shows whether changes genuinely improve performance
3	Fundamental data	Adds earnings, cash flow, valuation and balance-sheet evidence
4	Frontend	Makes the system usable and portfolio-ready
5	Monitoring and structured logs	Improves debugging and reliability after deployment
6	Hosted MySQL only if persistence is needed	Keeps user history across deployments

PROFESSIONAL POSITIONING

Describe the project as evidence-first stock research or technical decision support. Do not claim guaranteed predictions, professional financial advice or proven profitability.

12. Interview questions and strong answers

1. Explain the project in 30 seconds.

I built a hybrid stock-research backend using FastAPI. It collects live Yahoo Finance evidence, calculates reproducible technical signals, uses specialized Gemini agents for research and independent risk review, backtests the same rules, and exposes sources and calculation proof. I also added JWT authentication, SQLite persistence and background jobs for a future frontend.

2. Is the recommendation rule-based or AI-generated?

It is hybrid. Python rules lock the technical action and risk so they are reproducible and backtestable. AI agents prioritize research, challenge weaknesses and select the most useful supported evidence.

3. Why not let the LLM directly decide BUY or SELL?

An unrestricted LLM can hallucinate facts and return inconsistent answers. Financial calculations need traceability. I use deterministic code for numbers and constrained agents for interpretation.

4. What exactly do the agents do?

The Research Agent selects relevant company and news evidence, the Risk Agent challenges the signal and identifies gaps, and the Advisor Agent selects the most important supported strengths and warnings.

5. How do you prevent hallucinations?

Agents can return only allowed evidence IDs, news indexes, company fields and gap codes. The backend rejects unknown selections and constructs final factual wording from deterministic candidates.

6. How is the technical signal calculated?

Moving-average trend contributes plus or minus two, RSI contributes plus or minus one at configured boundaries, and MACD contributes plus or minus one. Scores at least two are BUY, at most minus two are SELL, and the remainder are HOLD.

7. What does evidence quality mean?

It measures data completeness and review coverage. It is intentionally not described as confidence in profit or prediction accuracy.

8. How did you avoid look-ahead bias in backtesting?

A signal uses information available at a day's close, but its simulated position starts on the following trading day. Transaction costs are also applied when positions change.

9. Why compare with buy-and-hold?

A strategy is not useful merely because it made money in a rising market. The benchmark shows whether the rules added value compared with simply owning the asset.

10. Why SQLite instead of MySQL now?

This is a single-user resume demo, so SQLite provides persistence without cost or infrastructure. SQLAlchemy keeps the code portable to MySQL through DATABASE_URL if deployment requirements change.

11. Why JWT authentication for a demo?

It demonstrates a real frontend-ready security flow and protects personal watchlists and saved history. Passwords are hashed with Argon2, and tokens expire.

12. Why are background jobs needed?

Agent workflows can be slow. The frontend receives a job ID immediately and polls status, which avoids a frozen interface and makes failures easier to present.

13. What happens when Gemini fails?

The deterministic calculation still exists. The analysis service falls back to a controlled narrative and marks agent_analysis.status as fallback instead of inventing an AI result.

14. What would you improve first?

I would run walk-forward tests across many tickers and market regimes, version the strategies, and compare out-of-sample performance before adding fundamental data and the frontend.

15. What was the hardest design decision?

Balancing intelligence with reliability. I kept the signal deterministic while constraining agents to verified evidence, which preserves both explainability and useful AI review.

13. Quick glossary and revision sheet

Term	Remember this
Ticker	Short market symbol such as AAPL
Adjusted close	Price corrected for corporate actions
MA	Moving average used to understand trend
RSI	0-100 momentum indicator; default warning zones are 70 and 30
MACD	Momentum difference between 12-day and 26-day exponential averages
Volatility	Historical variability of returns
Technical signal	Rule-based BUY/HOLD/SELL classification, not guaranteed advice
Agent	LLM role with a specific responsibility and restricted output
Hallucination	Plausible but unsupported AI-generated information
Backtest	Historical simulation of strategy rules
Look-ahead bias	Accidentally using future information in an earlier decision
Drawdown	Decline from a previous portfolio peak
JWT	Signed token used to identify an authenticated user
Hashing	One-way password protection
ORM	Database layer mapping Python models to tables
CORS	Browser rule controlling which frontend origins can call the API
Provenance	Where data came from and when it was collected

Five points to remember

- Yahoo Finance supplies evidence; Python calculates; agents interpret and challenge.
- The signal is reproducible because agents cannot change it.
- Proof explains the calculation; it does not guarantee future profit.
- Backtesting uses next-day execution and a buy-and-hold benchmark.
- SQLite and in-memory jobs are deliberate zero-cost demo choices, not accidental limitations.

RESUME DESCRIPTION

Built a hybrid multi-agent stock research platform combining live Yahoo Finance data, reproducible technical signals, vectorized backtesting, evidence-grounded Gemini agents, JWT authentication and auditable API responses.

For educational and informational research only. This project does not provide personalized financial advice or guarantee investment outcomes.